node (which is the key for the table). For the sake of simplicity, collisions resolution is not considered in this and the following figures.

Since the order is only based on insertion time, new elements are not changing the order of the older elements; therefore, we don't need anything fancy, we only need a structure that supports FIFO: we could just use a list, or a queue. As you should remember, a linked list is usually the best choice when we don't know in advance the number of elements we will have to store or this can change dynamically, while a queue is usually implemented using an array (and so more static in dimension), but optimized for insertion on the head and removal on the tail.

Linked lists as well can support fast insertion/removal at their ends: we need, however, a doubly-linked list where we insert elements on the front, remove from the tail; by always keeping a pointer to the tail and links from each node to its predecessor, we can implement tail removal in `O(1)` time.

More in depth, queues, being static and using arrays, save us some extra memory, that in linked lists needs to be used for pointers to other nodes (and for the node object itself). This is an implementation detail, though, that depends on the language used and, anyway, doesn't change the order of magnitude of the memory used: it's `O(n)` in both cases.

Listing 7.1 LRU Cache construction

```
class LRUCache
  #type integer
```